

Exercice : Conception et implémentation d'une base de données relationnelle avec SQLAlchemy

Thème : Gestion d'une Bibliothèque

Module : Bases de données avancées / Programmation orientée objet

Outil : SQLAlchemy (Python)

Durée estimée : 4 à 6 heures

1. Objectif

Mettre en pratique les trois types de relations en SQLAlchemy (One-to-One, One-to-Many, Many-to-Many) sur un nouveau domaine. L'objectif est de consolider la logique des relations entre modèles et de développer des compétences en conception de bases de données relationnelles.

2. Contexte

Vous devez concevoir une petite base de données pour gérer une bibliothèque. La base doit contenir exactement les trois types de relations relationnelles :

- One-to-Many (1:N)
- One-to-One (1:1)
- Many-to-Many (N:N)

3. Structure demandée

Vous devez créer les tables suivantes avec les relations indiquées :

Table	Description	Type de relation
Auteurs	Informations sur les auteurs de livres	—
Livres	Informations principales des livres	One-to-Many avec Auteurs
DetailsLivre	Informations complémentaires d'un livre (résumé, nombre de pages, note)	One-to-One avec Livres
Genres	Liste des genres littéraires	—
livres_genres	Table de liaison (association)	Many-to-Many (Livres ↔ Genres)

Règles métier

- Un auteur peut avoir écrit plusieurs livres (One-to-Many).
- Chaque livre a un seul auteur.
- Chaque livre possède un seul enregistrement de détails (One-to-One).
- Un livre peut appartenir à plusieurs genres, et un genre peut être associé à plusieurs livres (Many-to-Many).

4. Travail demandé

Vous devez réaliser les étapes suivantes :

1. Création des modèles SQLAlchemy

Définir les classes Auteur, Livre, DetailsLivre et Genre.

Implémenter correctement les 3 types de relations avec relationship().

Créer la table d'association livres_genres si nécessaire.

Ajouter les ForeignKey et les contraintes appropriées.

2. Configuration de la base de données

Créer un fichier de configuration de connexion (similaire à vos travaux précédents).

Créer une fonction init_db() qui crée toutes les tables.

3. Implémentation des opérations CRUD

Créer au minimum les fonctions suivantes :

- creer_livre(session)
- lister_livres(session)
- modifier_livre(session)
- supprimer_livre(session)

4. Utilisation des relations

Dans vos fonctions, manipuler les relations (ex: ajouter un genre à un livre, accéder à l'auteur via livre.auteur, etc.).

Utiliser l'eager loading (joinedload ou selectinload) dans au moins une requête de listing.

5. Application console (fortement recommandé)

Créer un petit menu interactif permettant de tester toutes les fonctionnalités CRUD.

5. Contraintes techniques

- Utiliser SQLAlchemy (version 1.4 ou 2.0 acceptée).
- Bien séparer le code (dossier dal/ recommandé : models.py, database.py, etc.).
- Les commentaires doivent être clairs et explicatifs.
- Le code doit être propre, lisible et bien structuré.
- Respecter les bonnes pratiques de nommage (PEP 8).

6. Livrables attendus

Un dossier compressé (.zip) contenant :

- Le dossier dal/ avec les fichiers models.py et database.py
- Le fichier main.py (ou console.py) avec le menu interactif
- Un fichier README.md expliquant la structure du projet et les relations implémentées

8. Questions à se poser (pour vous aider)

- Comment lier un livre à son auteur ?
- Comment faire en sorte qu'un livre ait automatiquement un enregistrement DetailsLivre ?
- Comment ajouter plusieurs genres à un livre sans dupliquer les données ?
- Quelle est la différence entre joinedload et selectinload dans ce contexte ?
- Comment gérer la suppression en cascade (si un livre est supprimé, que se passe-t-il pour ses détails ?) ?